Assignment 2 Report — VR Design Generation using Specifications

Course: CS6.402 – Engineering Virtual Reality Systems, Spring 2026
Student Roll Number: 2023111034
Scene Title: Unsealing the Tome of Dark Arts

---

1. Recorded Metrics

a. Total Time Taken

4 hours (from finalised requirements to final submission version).

b. Number of Development Iterations

3 iterations — Versions 4, 5, and 6.
(Versions 1–3 in the repository are pre-existing examples unrelated to this assignment. The work for this assignment begins at Version 4.)

c. Requirements Mismatched Per Iteration

#	Requirement (from SRS)	V4	V5	V6
1	Cauldron with states (Empty → Greened → Heated → Primed → Failed)	✅	✅	✅
2	Gas Valve with on/off state	Met	Met	Met
3	Green Vial: pour-then-empty behaviour	Met	Met	Met
4	Red Vial: pour-then-empty behaviour	Met	Met	Met
5	Strict brewing sequence with failure/explosion reset	Met	Met	Met
6	Iron Key: pick-up, drop in primed cauldron, transmute	Met	Met	Met
7	Bloodstone Key: spawns from transmutation, used to open Tome	Met	Met	Met
8	Tome: locked → open on correct key	Met	Met	Met
9	Prefab models for table, keys, and vials	Not met	Met	Met
10	Vial emptied (material swap to transparent) after pouring	Not met	Met	Met
11	Valve gives physical/visual feedback on interaction	Not met	Partially met	Met
12	Bloodstone Key floats in place (no physics drift)	Not met	Partially met	Met
13	Fire particle effect on burner ignition	Not met	Not met	Not met

#	Requirement (from SRS)	V4	V5	V6
14	Explosion/smoke particle effect on failure	Not met	Not met	Not met
15	Audio feedback (ignition loop, explosion sound, unlock cue)	Not met	Not met	Not met
16	Pouring animation for vials	Not met	Not met	Not met

Legend: Met | Partially met | Not met

Mismatch count per version: V4 — 8 mismatches | V5 — 4 mismatches | V6 — 4 mismatches (items 13–16 remain unachievable within current tool capabilities)

---

2. Challenges Encountered

Tool Capability Ceiling

The most significant challenge was the gap between what the SRS described and what VReqDV could natively support. The tool's behavior.json system is well-suited to state-machine driven interactions, but has no built-in support for particle systems, audio clips, or physics-animated transitions. Requirements 13–16 (particles, audio, pour animation) could not be met without modifying the tool, which was explicitly prohibited. These were acknowledged as permanent mismatches.

Prefab Pivot Offset and Physics

When the GoldKey.prefab was assigned to the BloodstoneKey article, the prefab's mesh pivot was not centred on the object origin. This caused two problems: (a) the Rigidbody component made it fall through the table on spawn, and (b) rotation-in-place using Transform.Rotate caused the object to drift in world space, because rotating about a non-centred local origin translates the bounding sphere. This was resolved by disabling the Rigidbody entirely and switching from Rotate() to directly assigning transform.eulerAngles using a static angle accumulator, which modifies only rotation without touching position.

Animated Valve Orbit

Version 5 implemented an instantaneous 90° self-rotation for the valve, which looked unconvincing. Version 6 replaced this with an orbit of the valve around the cauldron's Y-axis, animating at 90°/sec per frame via a ValveAnimating behavior that fires every frame while a static flag is set. The main challenge here was correctly resetting accumulated orbit state during an explosion reset, since a naïve -90° snap could be wrong if the valve was mid-animation. This was solved by tracking _valveOrbitCurrent and computing an exact correction delta.

Spawn Position in Code vs. Editor

The BloodstoneKey's reveal position was hardcoded in UserAlgorithms.cs. After the user repositioned scene assets in the Unity Editor to better fit the workbench prefab layout, the hardcoded spawn coordinates placed the key underground. This required reading the adjusted IronKey position from article.json and aligning the spawn point to match the same X/Z.

---

3. Reflection

Ease of Workflow

The VReqDV workflow — write JSON, hit "Display Mock-up", observe, iterate — is fast and low-friction for the first few iterations. Defining objects with states and wiring them to behaviors in JSON is more readable than setting up equivalent MonoBehaviour scripts from scratch, and the codegen removes a lot of boilerplate.

Clarity

The tool's separation of concerns (article = "what exists", behavior = "when things happen", UserAlgorithms = "how they happen") maps cleanly to the three-layer decomposition used in the SRS. For the first iteration it was straightforward to translate each rule from the SRS directly into a behavior entry. Where it broke down was when a requirement was inherently continuous (animation, particles) rather than discrete-event, because the OnCondition model requires a per-frame C# method rather than a declarative description.

Confidence in Correctness

The state-machine approach gives high confidence that the logic is correct: because each cauldron state (empty → greened → heated → primed) gates the next action, it is impossible to complete the sequence out of order. The main source of uncertainty was in the UserAlgorithms.cs code that manipulates transforms directly, where Unity's coordinate system and prefab pivot behaviour could cause subtle positional bugs — as seen with the valve and key issues above.

Overall Experience

The method is effective for rapid prototyping of interaction logic, but reaches its limits quickly when the SRS calls for perceptual richness (sound, particles, animation). For a purely logic-and-state prototype the tool is well-matched to the task. Production-quality implementation of the full SRS would require supplementing the generated code with hand-written MonoBehaviours for the presentation layer.

---

4. Unity Package

The exported Unity package is available at the following link:

Unity package link